

CSC311H5 Introduction to Machine Learning

Lecture 2

University of Toronto Mississauga

Review

Last class, we discussed

- Supervised learning: Learn a predictor given a **training set** consisting of **inputs** ($\mathbf{x} \in \mathbb{R}^D$) and the corresponding **target labels** t
- Having a separate validation set (for tuning hyperparameters) and test set (to assess model generalization)
- Our first supervised learning model: the k-Nearest Neighbour model

Today: Decision Trees!

A bit about machine learning discourse

When learning machine learning models, we always talk about **how to make predictions** before talking about **how to train our model**.

This may seem backwards: we train models *before* making predictions.

The order that we learn things will not be the order that we do things.

Section 1

Decision Trees

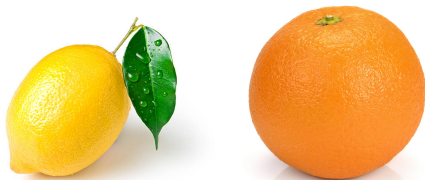
What is a decision tree model

- Like a series of nested if-else statements
- Simple but powerful learning algorithm
- Variations are widely used in ML competitions (e.g., Kaggle)
- Lets us motivate concepts from information theory (entropy, mutual information, etc.)

As is typical when discussing ML models, we will *discuss how to make predictions* first (i.e., **inference**), and *then* how to build models (i.e., **learning**).

Lemons or Oranges

Remember the citrus classification problem from lecture 1?

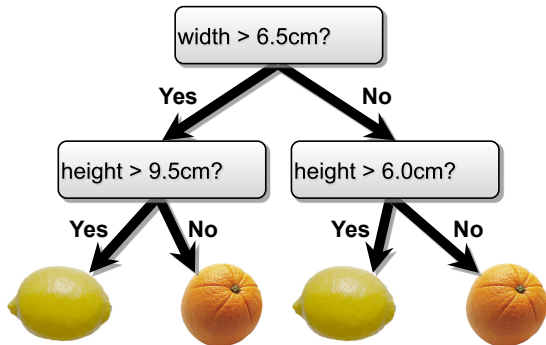


Scenario: You run a sorting facility for citrus fruits

- Binary classification: lemons or oranges
- Features measured by sensor on conveyor belt: height and width

Decision Trees

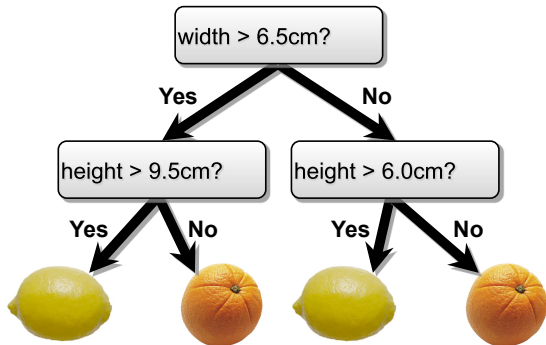
Make predictions by splitting on features according to a tree structure.



- **Internal nodes** test a *feature*
- **Branching** is determined by the *feature value*
- **Leaf nodes** are *output predictions*

Decision Trees

Make predictions by splitting on features according to a tree structure.

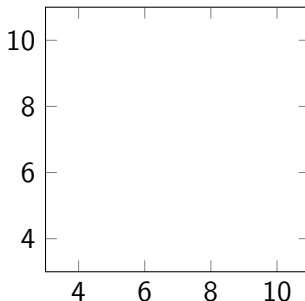
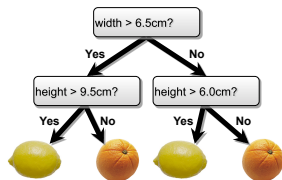


- **Internal nodes** test a *feature*
- **Branching** is determined by the *feature value*
- **Leaf nodes** are *output predictions*

Q: What would a citrus with width 6.4cm and height 10cm be classified?

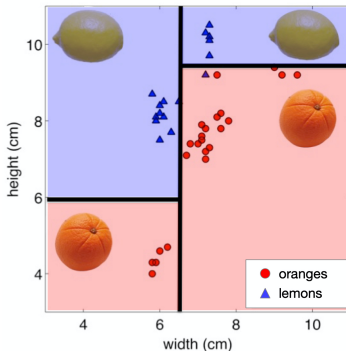
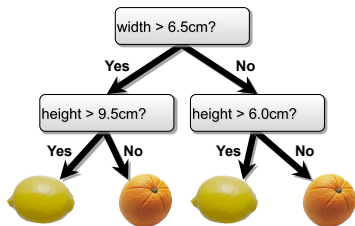
Visualizing Decision Tree Decision Boundary

Exercise: Draw the *decision boundary* for the decision tree on the left. Place the width measurement on the x-axis and the height measurement on the y-axis.



Decision Tree Decision Boundary

The decision boundary of a decision tree is made up of axis-aligned planes:



Decision Tree Output

Each path from root to a leaf defines a region R_m of input space, and points that fall into R_m will have prediction $y^{(m)}$. But what should this $y^{(m)}$ be?

Let $\{(\mathbf{x}^{(m_1)}, t^{(m_1)}), \dots, (\mathbf{x}^{(m_k)}, t^{(m_k)})\}$ be the training examples that fall into R_m

Classification Tree (Discrete Output)

Leaf value $y^{(m)}$ typically set to the **most common value** of t amongst training data points in R_m

Regression Tree (Continuous Output)

Leaf value $y^{(m)}$ typically set to the **mean value** of t amongst training data points in R_m

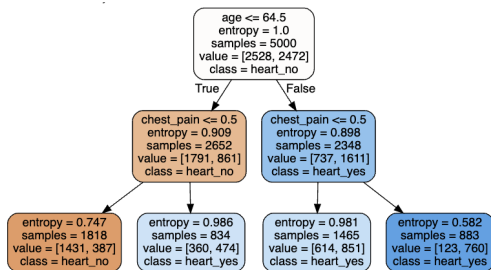
Example: HANES for Heart Disease Prediction

In labs 2, 5, and 7, we will use data from The National Health and Nutrition Examination Survey in the United States.

- Survey assess people's health and nutritional status
- Combines data from interviews and physical examinations
 - ▶ Race/Ethnicity
 - ▶ Ever had chest pain
 - ▶ Age
 - ▶ BMI
 - ▶ Blood pressure
 - ▶ ...
- Our Target: Presence of heart disease (self-reported)

In lab 2, we will build a *classification tree* to assess the presence of heart disease.

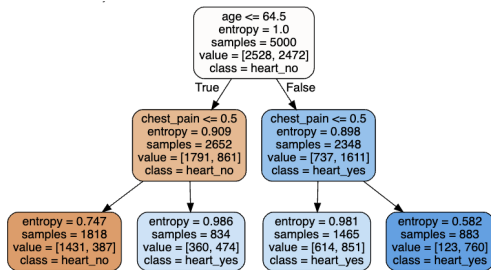
Decision Tree Features



Continuous Features can be partitioned into half-planes.

Example: AGE are partitioned.

Decision Tree Features

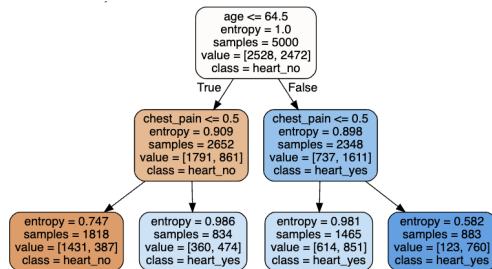


Discrete Features can be partitioned into its possible values.

Example: CHEST_PAIN is partitioned into its two possible values 0 and 1.

Exercise: Making a prediction (inference)

Suppose we have the following tree:



What prediction would we make for a person with:

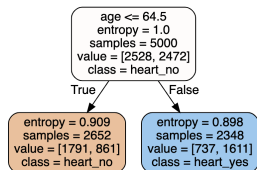
- Age: 64
- BMI 20.9
- No chest pain

Section 2

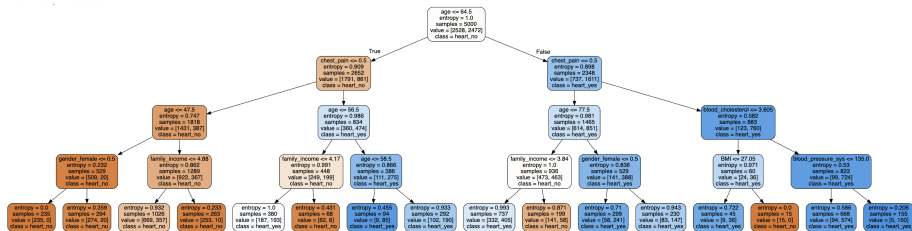
Constructing Decision Trees

Which decision tree is better?

A:



B:



Why constructing trees is hard

Decision trees are *universal function approximators*:

- For any training set we can construct a decision tree that has exactly the one leaf for every training point, but it probably won't generalize.
 - ▶ Example - If all D features were binary, and we had $N = 2^D$ unique training examples, a **Full Binary Tree** would have one leaf per example.
- Finding the smallest decision tree that correctly classifies a training set is NP complete.
 - ▶ If you are interested, check: Hyafil & Rivest'76.

So, how do we construct a useful decision tree?

Learning decision trees

We will use a **greedy heuristic**:

- Start with the whole training set and an empty decision tree.
- Pick a feature and candidate split that would most reduce a **loss**.
- Split on that feature and recurse on subpartitions.

Q: What would be a good loss to use? Ideas?

Learning decision trees

We will use a **greedy heuristic**:

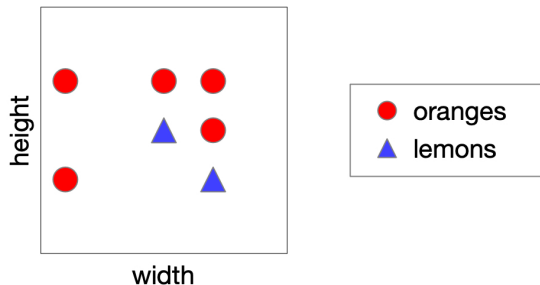
- Start with the whole training set and an empty decision tree.
- Pick a feature and candidate split that would most reduce a **loss**.
- Split on that feature and recurse on subpartitions.

Q: What would be a good loss to use? Ideas?

How about classification error or *misclassification rate*?

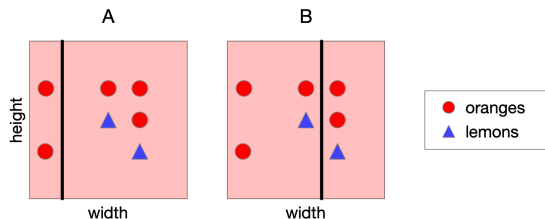
Example

Consider the following data. Let's split on width.



Split by misclassification rate?

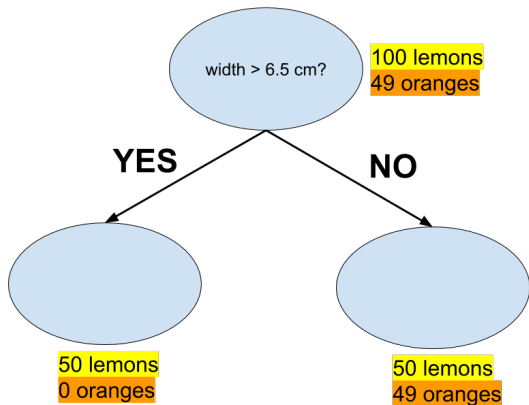
Let's try using **misclassification rate** as a loss.



Q: A and B have the same misclassification rate, so which is the best split?

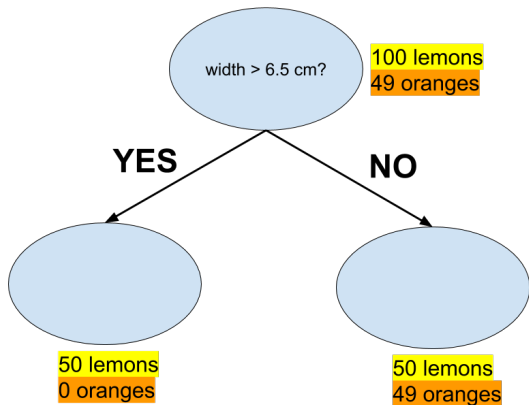
The problem with misclassification rate

Consider this split, which does not improve accuracy at all (compared to not having the split). Is this split good?



The problem with misclassification rate

Consider this split, which does not improve accuracy at all (compared to not having the split). Is this split good?



This split **reduces our uncertainty** about whether a fruit is a lemon!

Choosing a good split

How can we quantify uncertainty in prediction for a given leaf node?

- If all examples in leaf have same class: good, low uncertainty
- If each class has same amount of examples in leaf: bad, high uncertainty

Idea: Use counts at leaves to define probability distributions; use a probabilistic notion of uncertainty to decide splits.

In order to quantify uncertainty, we a brief detour through information theory. . .

Section 3

Information Theory

Entropy - Quantifying Uncertainty

You may have encountered the term **entropy** quantifying the state of chaos in chemical and physical systems,

In statistics, the **entropy** of a discrete random variable is a number that quantifies the **uncertainty** inherent in its possible outcomes:

$$H(X) = -\mathbb{E}_{X \sim p}[\log_2 p(X)] = -\sum_{x \in X} p(x) \log_2 p(x)$$

Intuitively, you can think of entropy as a number that represents how much certainty is **gained**, on average, by observing a random draw from a probability distribution.

Let's look at some examples. . .

Flipping Two Different Coins

Q: Which coin is more **uncertain**?

- First: 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, ...
- Second: 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, ...

Flipping Two Different Coins

Q: Which coin is more **uncertain**?

- First: 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, ...
- Second: 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, ...



Computing Entropy

Exercise What is the entropy of a loaded coin with probability $p = \frac{2}{18}$?
What about $p = \frac{10}{18}$?

Computing Entropy

Exercise What is the entropy of a loaded coin with probability $p = \frac{2}{18}$?
What about $p = \frac{10}{18}$?

First coin with $p = \frac{2}{18} = \frac{1}{9}$:

$$-\frac{8}{9} \log_2 \frac{8}{9} - \frac{1}{9} \log_2 \frac{1}{9} \approx \frac{1}{2}$$

Computing Entropy

Exercise What is the entropy of a loaded coin with probability $p = \frac{2}{18}$?
What about $p = \frac{10}{18}$?

First coin with $p = \frac{2}{18} = \frac{1}{9}$:

$$-\frac{8}{9} \log_2 \frac{8}{9} - \frac{1}{9} \log_2 \frac{1}{9} \approx \frac{1}{2}$$

Second coin with $p = \frac{10}{18} = \frac{5}{9}$:

$$-\frac{4}{9} \log_2 \frac{4}{9} - \frac{5}{9} \log_2 \frac{5}{9} \approx 0.99$$

Computing Entropy

Exercise What is the entropy of a loaded coin with probability $p = \frac{2}{18}$?
What about $p = \frac{10}{18}$?

First coin with $p = \frac{2}{18} = \frac{1}{9}$:

$$-\frac{8}{9} \log_2 \frac{8}{9} - \frac{1}{9} \log_2 \frac{1}{9} \approx \frac{1}{2}$$

Second coin with $p = \frac{10}{18} = \frac{5}{9}$:

$$-\frac{4}{9} \log_2 \frac{4}{9} - \frac{5}{9} \log_2 \frac{5}{9} \approx 0.99$$

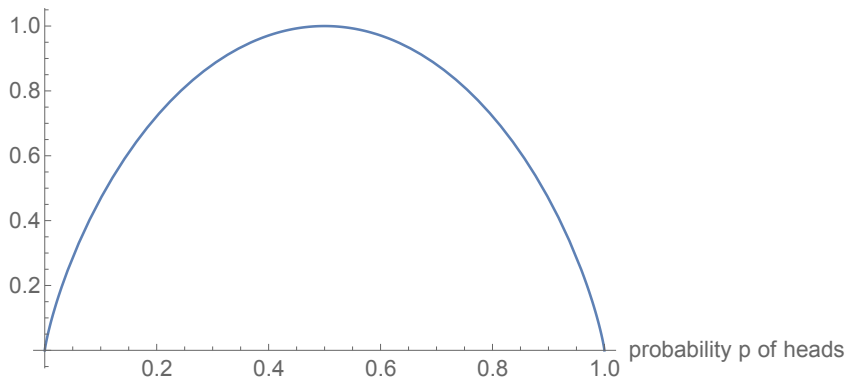
The coin whose outcomes are more certain has a lower entropy!

- In the extreme case $p = 0$ or $p = 1$, we were certain of the outcome before observing. So, we gained no certainty by observing it, i.e., entropy is 0.

Entropy of a Coin

The entropy of a coin is highest if the probability of obtaining heads is $p = \frac{1}{2}$

entropy



High vs Low Entropy Random Variables

High Entropy

- Variable has a uniform like distribution over many outcomes
- Flat histogram
- Values sampled from it are less predictable

Low Entropy

- Distribution is concentrated on only a few outcomes
- Histogram is concentrated in a few areas
- Values sampled from it are more predictable

Interpreting Entropy

We can also think of *entropy* as the expected information content of a random draw from a probability distribution.

Claude Shannon showed: you cannot store the outcome of a random draw using fewer expected **bits** than the entropy without losing information.

So units of entropy are **bits**; a fair coin flip has 1 bit of entropy.

Multiple Random Variables

Suppose we observe partial information X about a random variable Y

- Example: we want to know whether it is cloudy outside (Y) but can't see the sky, but we can see whether it is raining (X).

We want to work towards a definition of the expected amount of information that will be conveyed about Y by observing X .

Or equivalently, the expected reduction in our uncertainty about Y after observing X .

Entropy of a Joint Distribution

Example: $X = \{\text{Raining}, \text{Not raining}\}$, $Y = \{\text{Cloudy}, \text{Not cloudy}\}$

What is the entropy of system (X, Y) ?

	Cloudy	Not Cloudy
Raining	24/100	1/100
Not Raining	25/100	50/100

$$\begin{aligned}H(X, Y) &= -\mathbb{E}_{X, Y \sim p(x, y)}[\log_2 p(X, Y)] \\&= -\sum_{x \in X} \sum_{y \in Y} p(x, y) \log_2 p(x, y) \\&= -\frac{24}{100} \log_2 \frac{24}{100} - \frac{1}{100} \log_2 \frac{1}{100} - \frac{25}{100} \log_2 \frac{25}{100} - \frac{50}{100} \log_2 \frac{50}{100} \\&\approx 1.56 \text{ bits}\end{aligned}$$

Conditional Entropy

What is the entropy of cloudiness Y , **given that it is raining**?

	Cloudy	Not Cloudy
Raining	24/100	1/100
Not Raining	25/100	50/100

$$\begin{aligned} H(Y|X = \text{raining}) &= - \sum_{y \in Y} p(y|x = \text{raining}) \log_2 p(y|x = \text{raining}) \\ &= -\frac{24}{25} \log_2 \frac{24}{25} - \frac{1}{25} \log_2 \frac{1}{25} \\ &\approx 0.24 \text{ bits} \end{aligned}$$

We used: $p(y|x) = \frac{p(x,y)}{p(x)}$, and $p(x) = \sum_y p(x,y)$ (sum in a row)

Conditional Entropy and Expected Conditional Entropy

The **conditional entropy** is the entropy of the conditional distribution.

$$H(Y|X = x) = -\mathbb{E}_{Y \sim p(y|X=x)}[\log_2 p(y|X = x)]$$

This is what we computed in the previous slide.

The **expected conditional entropy** is defined as follows:

$$\begin{aligned} H(Y|X) &= \mathbb{E}_{x \sim p(x)}[H(Y|X = x)] \\ &= \sum_{x \in X} p(x) H(Y|X = x) \\ &= - \sum_{x \in X} \sum_{y \in Y} p(x, y) \log_2 p(y|x) \end{aligned}$$

Expected Conditional Entropy

What is the expected entropy of cloudiness, given the knowledge of whether or not it is raining?

	Cloudy	Not Cloudy
Raining	24/100	1/100
Not Raining	25/100	50/100

$$\begin{aligned} H(Y|X) &= \sum_{x \in X} p(x) H(Y|X = x) \\ &= \frac{1}{4} H(Y|X = \text{is raining}) + \frac{3}{4} H(Y|X = \text{not raining}) \\ &\approx 0.75 \text{ bits} \end{aligned}$$

Properties of Entropy and Conditional Entropy

- H is always non-negative
- Chain rule: $H(X, Y) = H(X|Y) + H(Y) = H(Y|X) + H(X)$
- If X and Y independent, then X does not affect our uncertainty about Y : $H(Y|X) = H(Y)$
- By knowing Y makes our knowledge of Y certain: $H(Y|Y) = 0$
- By knowing X , we can only decrease uncertainty about Y :
 $H(Y|X) \leq H(Y)$

Information Gain

How much more certain am I about whether it's cloudy if I'm told whether it is raining? My uncertainty in Y minus my expected uncertainty that would remain in Y after seeing X .

This is called **information gain** $IG(Y|X)$ in Y due to X , or the **mutual information** of Y and X

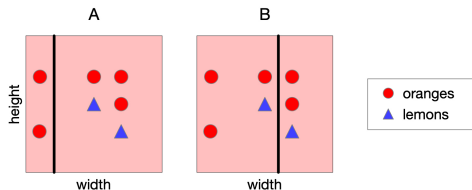
$$IG(Y|X) = H(Y) - H(Y|X)$$

- If X is completely uninformative about Y : $IG(Y|X) = 0$
- If X is completely informative about Y : $IG(Y|X) = H(Y)$

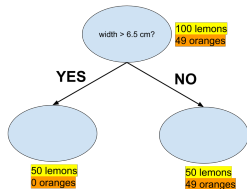
How does this help with decision trees?

In the case of decision trees, Y is the output class (e.g., lemon or orange), X is which side of the split a data point is in (e.g., left or right).

Q: Which is the better split?



Q: Is this a good split?

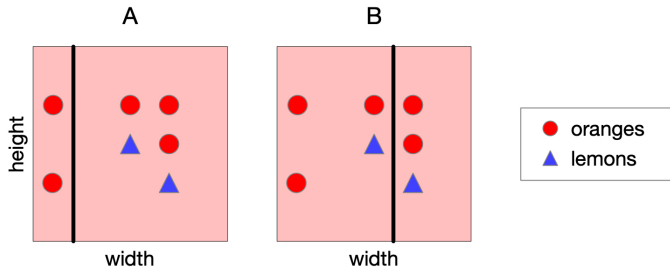


An alternative to misclassification rates

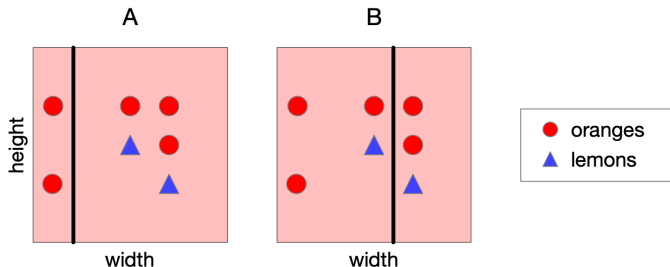
We will use **information gain** to quantify how “good” a split is.

- **Entropy** $H(Y)$ [bits]: characterizes the uncertainty in a draw of a random variable
- **Conditional Entropy** $H(Y|X)$ [bits] : characterizes the uncertainty in a draw of Y after observing X
- **Information gain** measures the informativeness of a variable, which is exactly what we desire in a decision tree split!
- The **information gain of a split**: how much information (over the training set) about the class label Y is gained by knowing which side of a split you're on.

Exercise: What is the information gain of split B?

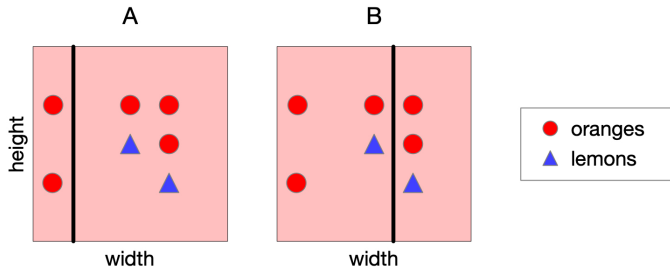


Exercise: What is the information gain of split B?

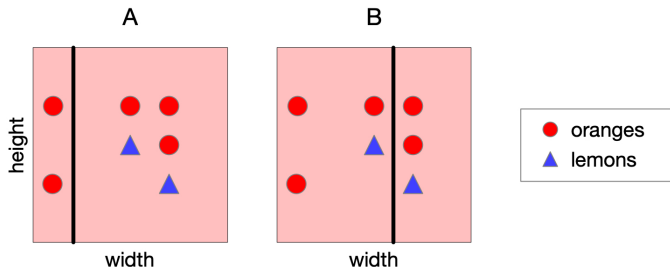


- Root entropy of class outcome:
 $H(Y) = -\frac{2}{7} \log_2(\frac{2}{7}) - \frac{5}{7} \log_2(\frac{5}{7}) \approx 0.86$
- Leaf conditional entropy of class outcome: $H(Y|left) \approx 0.81$,
 $H(Y|right) \approx 0.92$
- $IG(split) \approx 0.86 - (\frac{4}{7} \cdot 0.81 + \frac{3}{7} \cdot 0.92) \approx 0.006$

Exercise: What is the information gain of split A?



Exercise: What is the information gain of split A?



- Root entropy of class outcome:
 $H(Y) = -\frac{2}{7} \log_2\left(\frac{2}{7}\right) - \frac{5}{7} \log_2\left(\frac{5}{7}\right) \approx 0.86$
- Leaf conditional entropy of class outcome: $H(Y|left) = 0$,
 $H(Y|right) \approx 0.97$
- $IG(split) \approx 0.86 - \left(\frac{2}{7} \cdot 0 + \frac{5}{7} \cdot 0.97\right) \approx 0.17!!$

Section 4

Learning Decision Trees (Continued)

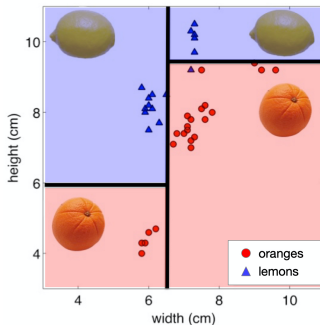
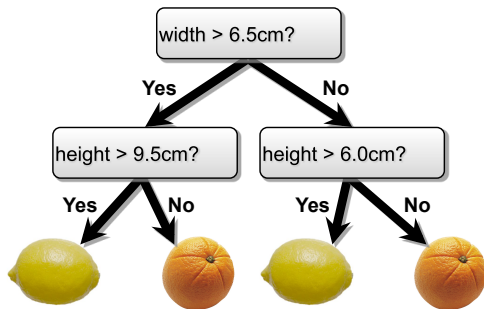
Constructing Decision Trees

At each level, we choose:

- Which feature to split
- Possibly where to split it (e.g., for a continuous feature)

Choose feature and split that provides the **highest information gain**.

Visualizations to keep in mind



Decision Tree Construction Algorithm

Recall the **greedy heuristic** we proposed earlier:

- Start with the whole training set and an empty decision tree.
- Pick a feature and candidate split that would most reduce a **loss**.
 - ▶ e.g., information gain!
 - ▶ potentially square loss for a regression tree.
- Split on that feature and recurse on subpartitions.

Discussion

Q: When might we wish to stop splitting nodes?

Discussion

Q: When might we wish to stop splitting nodes?

Possible answer: Terminates when all leaves contain only examples in the same class or are empty.

Possible answer: When a depth limit is reached, or when the loss stops improving much

Handling Continuous Attributes

- Split based on a threshold, chosen to maximize information gain
 - ▶ Since the training data is finite, there are only a finite number of thresholds to consider!
- Decision trees can also be used for regression on real-valued outputs.
 - ▶ Choose splits to minimize squared error, rather than maximize information gain.

What Makes a Good Tree?

Not too small:

- need to handle important but possibly subtle distinctions in data

Not too big:

- Computational efficiency (avoid redundant, spurious attributes)
- Avoid over-fitting training examples
- Human interpretability

Occam's Razor find the simplest hypothesis that fits the observations

- Useful principle, but hard to formalize (how to define simplicity?)
- We desire small trees with informative nodes near the root

Decision Tree Miscellany

Problems:

- You have exponentially less data at lower levels
- Too big of a tree can overfit the data
- Greedy algorithms don't necessarily yield the global optimum

There are other criteria used to measure the quality of a split, e.g., Gini index

Trees can be pruned in order to make them less complex

Decision trees can also be used for regression on real-valued outputs.

- Choose splits to minimize squared error, rather than maximize information gain.

Section 5

Notations and Indicator Variables

Example: HANES for Heart Disease Prediction

In labs 2, 5, and 7, we will use data from The National Health and Nutrition Examination Survey in the United States.

- Survey assess people's health and nutritional status
- Combines data from interviews and physical examinations
 - ▶ Race/Ethnicity
 - ▶ Ever had chest pain
 - ▶ Age
 - ▶ BMI
 - ▶ Blood pressure
 - ▶ ...
- Our Target: Presence of heart disease (self-reported)

Q: Why is this learning setup potentially problematic?

Machine Learning Pipeline

- Collecting and understanding the data
- Clean and prepare the data
 - ▶ Determining the features to use
 - ▶ Splitting into training/validation/test sets
- Model training & hyperparameter tuning
- Evaluating generalization accuracy
- Additional testing (e.g., subgroup analysis, adversarial testing)

Supervised Learning Notation (NHANES)

Input: Represented using the vector $\mathbf{x}$ containing features for a single data point

Target Output: Represented using the scalar $t \in \{0, 1\}$

Goal: We wish to learn a decision tree $t \approx y = f(\mathbf{x})$, $f : \mathbb{R}^n \mapsto \mathbb{R}$.

- Data: $(\mathbf{x}^{(1)}, t^{(1)}), (\mathbf{x}^{(2)}, t^{(2)}), \dots (\mathbf{x}^{(N)}, t^{(N)})$
- The $\mathbf{x}^{(i)}$ are the *inputs*
- The $t^{(i)}$ are the *targets* (or the *ground truth*)

NHANES Heart Disease Prediction

Suppose that for the NHANES heart disease prediction problem, we use $d = 4$ features (simplified! in the lab we will use more!):

- The numerical age feature
- The numerical BMI feature
- The numerical blood_pressure_sys feature
- The numerical diastolic_bp feature

Then one data point from the training set $(\mathbf{x}^{(1)}, t^{(1)})$ will look like this:

$$\mathbf{x}^{(1)} = \begin{bmatrix} 62 \\ 29.10 \\ 104 \\ 56 \end{bmatrix}, \quad t^{(1)} = 0$$

Data Matrix

We can express computations more succinctly by using linear algebra notation.

Start by putting the entire data set in a single, **data matrix**

$$\mathbf{X} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_D^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_D^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(N)} & x_2^{(N)} & \dots & x_D^{(N)} \end{bmatrix}$$

Data Matrix

We can express computations more succinctly by using linear algebra notation.

Start by putting the entire data set in a single, **data matrix**

$$\mathbf{X} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_D^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_D^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(N)} & x_2^{(N)} & \dots & x_D^{(N)} \end{bmatrix}$$

Q: What is the **shape** of the data matrix?

Data Matrix

We can express computations more succinctly by using linear algebra notation.

Start by putting the entire data set in a single, **data matrix**

$$\mathbf{X} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_D^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \dots & x_D^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(N)} & x_2^{(N)} & \dots & x_D^{(N)} \end{bmatrix}$$

Q: What is the **shape** of the data matrix?

$N \times D$, where N is the number of data points in the training data, and D is the number of features.

Vectorizing Targets and Predictions

Likewise, we can fold the entire set of target values in a single vector, and an entire set of predictions in a single vector.

$$\mathbf{t} = \begin{bmatrix} t^{(1)} \\ t^{(2)} \\ \vdots \\ t^{(N)} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(N)} \end{bmatrix}$$

Representing features

Each numerical features is represented as a column in the data matrix.

But what about categorical features?

- Gender (recorded as binary in NHANES): 1=female, 0=male
- Race Ethnicity: 1=hispanic, 2=white, 3=black, 4=asian, 5=other
- ...

Representing Binary Features

For binary features, we can use **indicator features**.

For example, for the gender feature, $x_{gender}^{(i)} = 1$ if and only if the i th data point represents a person recorded as being “female”.

Representing Categorical Features

What about for categorical features with many categories?

- Race Ethnicity: 1=hispanic, 2=white, 3=black, 4=asian, 5=other

Q: Can we treat categorical features as numerical?

Representing Categorical Features

What about for categorical features with many categories?

- Race Ethnicity: 1=hispanic, 2=white, 3=black, 4=asian, 5=other

Q: Can we treat categorical features as numerical?

This approach can be problematic! What would decision tree splits like `ethnicity >= 2.5` mean?

Indicator Features (Indicator Variables, One-Hot Encoding)

Each category in the categorical variable becomes its own feature.

Instead of a single feature “race_ethnicity”, we will have features:

- `ethnicity_hispanic` $\in \{0, 1\}$
- `ethnicity_white` $\in \{0, 1\}$
- `ethnicity_asian` $\in \{0, 1\}$
- `ethnicity_black` $\in \{0, 1\}$

This approach makes it easier for decision trees to create split based on specific ethnicities.

What you will do in Lab 2

- Explore the NHANES data set
- Create the data matrix; split the train/validation/test sets
- Train and visualize several decision tree classifiers
- Explore the generalization performance